DT-Drive: A Tool for Deterministic Replay-Based Testing and Debugging of Autonomous Driving Systems

Sanjeetha Pennada
University of Sheffield
School of Computer Science
Sheffield, United Kingdom
s.pennada@sheffield.ac.uk

Matthew Leach
University of Sheffield
School of Computer Science
Sheffield, United Kingdom
m.leach@sheffield.ac.uk

Olek Osikowicz
University of Sheffield
School of Computer Science
Sheffield, United Kingdom
amosikowicz1@sheffield.ac.uk

Donghwan Shin
University of Sheffield
School of Computer Science
Sheffield, United Kingdom
d.shin@sheffield.ac.uk

Abstract

While simulation is essential for testing autonomous driving systems (ADS), high-fidelity simulators like CARLA often exhibit non-determinism, leading to flaky simulation results. We present DT-Drive, a record-modify-replay framework that ensures deterministic ADS evaluation by decoupling the ego vehicle from the recorded driving scenario. This approach enables "ego-injection" for fair multi-ADS comparisons and "counterfactual replay" via environmental modifications (e.g., weather and traffic), facilitating targeted testing and debugging. Validated on 128 CARLA benchmark scenarios, DT-Drive achieved 100% deterministic evaluation results, effectively eliminating simulator-induced flakiness. We also demonstrate its utility for testing and debugging by conducting a controlled comparison of the TransFuser and TransFuser++ agents under identical and modified environmental conditions.

DT-Drive and the sample data are available at https://github.com/SanjeethaPennada/DT-Drive [9]. The tool demo video is available at https://youtu.be/HB1DIZSF5E0.

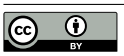
CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; **Software functional properties**; • **Computer systems organization** → **Embedded and cyber-physical systems**; • **Computing methodologies** → **Simulation evaluation**.

Keywords

Autonomous Driving Systems, Simulation-based Testing, Record and Replay, Flaky Tests

ACM Reference Format:
Sanjeetha Pennada, Matthew Leach, Olek Osikowicz, and Donghwan Shin. 2026. DT-Drive: A Tool for Deterministic Replay-Based Testing and Debugging of Autonomous Driving Systems. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26), October 12–16, 2026, Munich, Germany.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3832783.3834601



This work is licensed under a Creative Commons Attribution 4.0 International License.
ASE '26, Munich, Germany
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2882-2/2026/10
https://doi.org/10.1145/3832783.3834601

1 Introduction

Rigorous testing of Autonomous driving systems (ADS) across a wide range of driving scenarios is essential to ensure their safety and reliability. A test scenario defines a specific sequence of events that the ADS must handle, including both *static* entities (e.g., road layouts, traffic lights, and buildings) and *dynamic* entities (e.g., surrounding vehicles and pedestrians). By executing the ADS in such environments, developers can verify whether the system violates safety requirements, such as collisions with other vehicles or pedestrians [10]. Due to the high cost, risks, and limited controllability of real-world testing, simulation-based testing has become a widely adopted approach for evaluating ADS performance [12]. However, high-fidelity and widely used driving simulators, such as CARLA [4], do not guarantee deterministic execution [7, 8]; for example, Osikowicz et al. [8] empirically revealed that 31.3% (40 out of 128) of the CARLA benchmark scenarios exhibit potentially flaky behaviour, meaning identical scenarios can produce different safety evaluation outcomes across repeated runs, even when ADS is deterministic.

Record-and-replay (RnR) is a well-established paradigm for enhancing the reproducibility of testing and debugging complex software systems [2]. By capturing runtime events, deterministic replay frameworks enable developers to execute systems offline while strictly preserving their original behaviour [11]. This capability is particularly valuable for isolating transient faults, allowing engineers to repeatedly analyse executions to diagnose bugs that are otherwise extremely difficult to reproduce [5]. Given this proven utility, one might assume that the native RnR functionality provided by modern driving simulators like CARLA could naturally resolve the issue of simulator-induced flakiness. However, while CARLA's native mechanism successfully guarantees a deterministic replay of a previous run, it operates as an immutable playback loop. Crucially, it lacks the architectural flexibility to decouple the recorded scenario from the autonomous driving agent. Consequently, users cannot replace the ADS controlling the ego vehicle during replay, severely restricting the ability to benchmark multiple, distinct ADSs under

Table 1: Key functionalities of CARLA built-in Record-and-Replay (RnR) and DT-Drive

Feature	CARLA native RnR	DT-Drive
Record and replay	Recording and replaying a driving scenario simulation. Once a recording is generated, replaying it is fully deterministic.	
Ego-injection replay	Not supported.	Swapping the current ADS agent during playback to evaluate different systems under identical conditions. Enables fully deterministic multi-ADS comparison.
Counterfactual replay	Not supported.	Modifying recorded environmental entities (e.g., weather, NPCs) to test "what-if" scenarios. Enables follow-up testing and debugging.

fully identical environmental conditions. Furthermore, the native RnR system does not support the targeted modification of recorded environmental entities or surrounding traffic. This limitation prevents researchers from conducting counterfactual, "what-if" analyses, such as systematically altering weather variables or removing specific obstacles, which are vital for testing ADS robustness and isolating the causes of safety failures.

To address the challenges, we propose DT-Drive, a record-modify-replay framework for simulation-based ADS testing allowing deterministic replays. DT-Drive can be easily integrated into existing scenario-generation studies. Our framework decouples recorded scenarios from the ego vehicle controller, enabling an injection of the new driving agent during replay while guaranteeing deterministic execution. In addition, it supports counterfactual replay, enabling researchers to systematically vary environmental conditions while preserving reproducibility. Table 1 compares DT-Drive with CARLA's native Record and Replay (RnR).

We evaluate DT-Drive using 128 scenarios from prior work on simulator flakiness [8], implemented in CARLA [4], with TransFuser++ [3] as the ADS under test. Our results demonstrate that DT-Drive eliminates execution variability observed in standard simulator runs, producing consistent outcomes across repeated executions. Furthermore, we show that DT-Drive enables controlled and fair comparison of multiple ADSs by evaluating both TransFuser++ and TransFuser under identical deterministic scenarios as well as modified simulation environments. In summary, this paper makes the following contributions:

- We propose DT-Drive, a novel record–modify-replay framework that (a) guarantees deterministic scenario execution, (b) enables interchangeable evaluation of multiple ADSs under identical conditions, and (c) supports controlled scenario modification for systematic experimentation.
- We demonstrate that DT-Drive eliminates simulation flakiness and enables reproducible, fair comparisons of ADSs across both deterministic and modified scenarios. We also show that DT-Drive can be effectively integrated into both scenario-generation and analysis workflows, enabling reliable, reproducible evaluation of multiple ADSs under identical or controlled conditions.

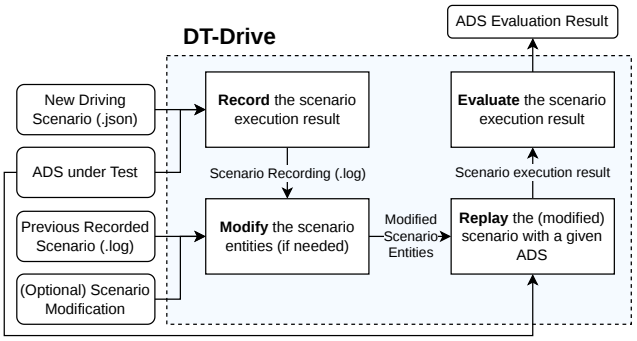


Figure 1: DT-Drive Overview. DT-Drive can start from a new scenario description or an existing scenario previously recorded by CARLA's native recorder.

2 DT-Drive Framework

Figure 1 shows an overview of DT-Drive. As an input it takes a scenario definition to run (either a new or previously recorded scenario), the ADS under test, and optional scenario modifications (e.g. changing static/dynamic entities). It then returns a deterministic ADS evaluation result (e.g., driving score) for a given scenario. Internally, DT-Drive operates through four major stages: *recording* the scenario, *modifying* entities, *replaying* the simulation with a target ADS, and *evaluating* the results.

2.1 Stage 1: Record

This stage begins by executing a test scenario (.json) in CARLA, which configures the environment's *static* road layouts and *dynamic* actors, such as pedestrians and surrounding vehicles. While the ADS agent controls the ego vehicle, CARLA's native recorder captures the states and interactions of all entities to produce a deterministic binary log (.log). By utilising this native format, DT-Drive can seamlessly ingest new or existing recordings for subsequent modification and replay.

2.2 Stage 2: Modify

This stage enables *counterfactual replay* by allowing users to adjust environmental conditions and entity behaviours. DT-Drive treats each static and dynamic entity as an independent dimension in a scenario vector representation, enabling precise, isolated modifications to the original recording. This capability facilitates "what-if" testing, which is essential for debugging complex interactions and assessing ADS robustness by independently varying scenario elements before re-executing the simulation.

Since the recorded log file is stored in a binary format and is not directly editable, DT-Drive cannot generate a modified execution trace by editing the log file itself. Instead, the specified modifications are applied during the replay initialization phase (Section 2.3), and the simulation is re-executed to generate a new deterministic execution that reflects the modified scenario.

2.3 Stage 3: Replay

This stage ensures deterministic execution by configuring CARLA in synchronous mode with a fixed simulation time step. Its primary function is *ego-injection*, where the original ego vehicle is replaced by the ADS under test at identical initial coordinates. While other entities follow trajectories replayed from the log, the ADS interacts with the environment in a closed loop. Because of this closed-loop interaction, execution time is comparable to that of a standard CARLA simulation, unlike the typically faster, open-loop native playback. This setup facilitates a consistent, multi-ADS comparison under identical conditions, allowing developers to isolate performance variances and detect non-deterministic software behaviours.

A key technical challenge is ensuring that the replay remains deterministic while replacing the recorded ego vehicle with the ADS under test and applying scenario modifications (if any). DT-Drive addresses this by pausing the simulation before the first simulation tick to establish a deterministic initial state. During this initialisation phase, the specified environmental modifications are applied, the original ego vehicle is replaced with the ADS under test, and the required sensors and control interfaces are attached. The simulation is then resumed in synchronous mode, ensuring that replayed actors continue from their recorded states while the ADS assumes full control of the ego vehicle. By performing all initialisation before execution begins, DT-Drive preserves replay determinism and guarantees that behavioural differences arise solely from the ADS or the injected scenario modifications.

2.4 Stage 4: Evaluate

In the final stage, DT-Drive generates a deterministic ADS evaluation result by calculating a Driving Score based on CARLA Leaderboard standards [1]. This score synthesises route completion with critical safety infractions, such as collisions, traffic violations, and lane departures, to provide a robust performance benchmark. These metrics are exported as a .json file, allowing developers to analyse agent behaviour and verify safety requirements across identical or modified conditions. Users can easily modify the evaluation metrics by updating the `StatisticsManager` class in `leaderboard/leaderboard/utils/statistics_manager.py`.

3 Validation Case Studies

3.1 Experimental Setup

We validated DT-Drive using 128 scenarios from the empirical study by Osikowicz et al. [8], which identified simulator non-determinism in 31.3% of the CARLA 0.9.10.1 Leaderboard benchmark[1] when executed with the TransFuser++ ADS [6]. These scenarios, known to produce flaky outcomes across repeated runs, serve as a baseline to test DT-Drive's ability to ensure determinism. It is measured against the driving score to verify if DT-Drive eliminates the evaluation flakiness inherent in standard CARLA executions.

3.2 Case Study 1: Deterministic Replay

We replayed the 128 scenarios with DT-Drive, including 40 previously identified as flaky due to simulator non-determinism [8], following the same methodology of Osikowicz et al. [8]. While the original study reported flaky test outcomes, DT-Drive produced identical results for all repeated runs as expected. This demonstrates that replaying actor trajectories via DT-Drive effectively eliminates simulator-induced flakiness and enables reproducible assessment of ADS performance.

[1]Support for the latest CARLA version is planned for a future update.

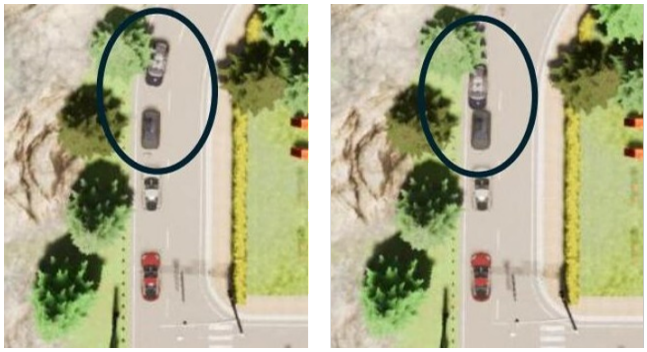


Figure 2: Ego-injection replay results. TransFuser++ (left) safely stopped when obstructed by a vehicle, whereas TransFuser (right) collided with the lead car.

3.3 Case Study 2: Ego-Injection Replay

We demonstrated DT-Drive's ego-injection capability by evaluating *Scenario 36*, one of the 128 scenarios, using two different ADSs, TransFuser [3] and TransFuser++ [6], under identical deterministic conditions. By replaying fixed actor trajectories, DT-Drive enabled a controlled comparison of their differing behaviours: TransFuser++ safely stopped when obstructed by a vehicle, whereas TransFuser collided with the lead car (Figure 2). Each test was repeated ten times, yielding 100% consistent outcomes. This confirms that DT-Drive facilitates fair, reproducible benchmarking by ensuring that variations in evaluation results stem solely from the ADS logic rather than simulation flakiness.

3.4 Case Study 3: Counterfactual Replay

To demonstrate counterfactual replay, we modified *Scenario 36* by removing street lights within a 20m radius and transitioning the time of day to night to reduce camera visibility. Under these modified conditions, TransFuser++ remained safely stopped, consistent with its original behaviour. However, TransFuser avoided the collision observed in the baseline run (Figure 3). Ten repetitions for each ADS yielded identical results, suggesting that TransFuser's earlier collision was sensitive to visibility conditions. This demonstrates that DT-Drive's counterfactual replay facilitates follow-up testing and debugging by allowing researchers to isolate environmental factors while maintaining strict determinism.

3.5 Integration with Existing Testing Workflows

DT-Drive integrates seamlessly with scenario generation and analysis frameworks, providing a deterministic layer without altering established workflows. By leveraging native simulator recording (see Section 2), the framework can ingest execution traces from any generation pipeline; only a few lines of instrumentation are needed to create a recording log during scenario execution. Once



Figure 3: Counterfactual replay results. TransFuser++ (left) maintains its baseline behaviour despite reduced visibility, whereas TransFuser (right) avoids the collision observed in the original scenario.

the log file (recording) is provided, users can perform ego injection, apply counterfactual modifications, and maintain original route definitions to ensure a fair, reproducible evaluation using DT-Drive.

This compatibility facilitates diverse research directions, such as scenario simplification [10], where strict determinism is required to verify that behavioural shifts stem solely from controlled modifications rather than simulator noise. Consequently, DT-Drive serves as a general-purpose utility that adds reliability and flexibility to a wide range of ADS testing methodologies, from critical scenario generation to automated debugging.

4 Discussion

4.1 Additional Findings from Case Studies

Non-determinism in Post-Collision. While DT-Drive guarantees deterministic execution for the majority of the simulation, we identified a limitation regarding the ego vehicle's post-collision physical dynamics. In Scenarios 2 and 29, for instance, the ADS acted deterministically and maintained identical trajectories until impact. However, the subsequent rigid-body dynamics varied slightly; in some iterations, the vehicle flipped and recovered, while in others, it remained inverted. These discrepancies stem from the simulator's physics engine. Therefore, the resulting evaluation metrics may diverge if the simulation is allowed to continue after the first collision. This highlights that while DT-Drive isolates ADS behaviour from environmental noise, it cannot fully override the inherent nondeterminism of the underlying physics engine during collisions.

Managing Flaky Behaviour Variants. DT-Drive captures one instance of multiple flaky behaviours into a deterministic record. For example, without the framework, *Scenario 8* exhibited five distinct behaviours across ten runs. Using DT-Drive, a developer can generate five unique logs, each of which can be selected for deterministic replay and analysis. This is particularly useful for debugging simulator errors; for example, one can replay individual logs, identify potential simulation bugs (e.g. a car passing through a static barrier [8]), and effectively remove them from the subsequent ADS testing and debugging. However, distinguishing between physically valid traces and subtle simulation artefacts in general remains an open research challenge.

4.2 Practical Takeaways

DT-Drive provides a practical solution for improving the reliability of simulation-based ADS testing. Based on our findings, we highlight the following key takeaways for researchers and practitioners:

- **Deterministic Debugging:** Developers can reliably reproduce failures by recording and deterministically replaying scenarios, eliminating ambiguity caused by simulator-induced variability.
- **Fair ADS Comparison:** By decoupling scenarios from the ego controller, DT-Drive allows multiple ADSs to be evaluated under identical conditions. This supports rigorous benchmarking and regression testing.
- **Reduced Computational Overhead:** Instead of repeatedly executing the same scenario to account for nondeterminism, practitioners can record only one representative execution and replay it deterministically. This significantly reduces testing cost.
- **Controlled Experimentation:** The framework enables systematic modification of scenarios while preserving determinism, facilitating robustness testing and debugging.

4.3 Limitations

Despite its advantages, DT-Drive has a few limitations.

First, it relies heavily on the CARLA simulator, which makes it harder to apply the method to other simulators. However, CARLA is the leading open-source simulator in autonomous driving community and is extensively adopted by both academia and industry. As a result, supporting CARLA enables DT-Drive to benefit a large portion of the ADS testing community, providing practical impact.

Second, it deterministically replays a single execution trace at a time, still requiring multiple runs to capture the full range of flaky behaviours, if any. Furthermore, because DT-Drive operates on recorded simulator executions, it cannot correct inherent simulation inaccuracies; instead, it only reproduces them deterministically. Nevertheless, focusing on one deterministic behaviour allows us to properly analyse it and debug the ADS under test (or the simulator) if necessary, making simulation-based ADS testing more reliable.

Finally, replaying scenarios recorded with one ADS using a different ADS may introduce unavoidable collisions. This occurs because NPC trajectories remain fixed independently of another ADS's behaviour. Although this could create unrealistic collisions, it is necessary to ensure fully deterministic testing results across multiple ADSs. Its potential impact on the realism of testing results deserves further study.

5 Conclusion

While simulation is a cornerstone of ADS evaluation, its reliability is frequently undermined by simulator induced non-determinism. This flakiness produces inconsistent outcomes for identical scenario configurations, complicating the validation of safety-critical systems. In this paper, we introduced DT-Drive that ensures reproducible testing by decoupling the ego vehicle from the recorded environment. By enforcing deterministic simulator settings and utilising a record-modify-replay approach, DT-Drive maintains identical actor trajectories and environmental conditions across repeated runs.

Our evaluation across 128 previously identified flaky scenarios demonstrates that DT-Drive effectively eliminates performance variability, yielding 100% consistent results. Furthermore, our case studies confirm that the framework facilitates high-fidelity "what-if" analysis through counterfactual replay and enables fair, ADS-agnostic benchmarking through ego-injection replay. Currently, the tool is implemented using an earlier version of the CARLA simulator, which fully supports the functionality required for our experiments. Migrating DT-Drive to the latest CARLA release is part of our future work to ensure compatibility with ongoing simulator development.

Acknowledgments

This work was supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No. RS-2025-02218761, 50%) and by the Engineering and Physical Sciences Research Council (EPSRC) [EP/Y014219/1].

Data Availability

The replication package with installation steps is available [9].

References

[1] CARLA Team. 2020. Get started with Leaderboard 1.0. http://leaderboard.carla.org/get_started_v1_0/. Online; accessed 2026.

[2] Yunji Chen, Shijin Zhang, Qi Guo, Ling Li, Ruiyang Wu, and Tianshi Chen. 2015. Deterministic replay: A survey. *ACM Computing Surveys (CSUR)* 48, 2 (2015), 1–47. doi:10.1145/2790077

[3] Kashyap Chitta, Aditya Prakash, Bernhard Jaeger, Zehao Yu, Katrin Renz, and Andreas Geiger. 2023. TransFuser: Imitation With Transformer-Based Sensor Fusion for Autonomous Driving. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 11 (2023), 12878–12895. doi:10.1109/TPAMI.2022.3200245

[4] Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. 2017. CARLA: An open urban driving simulator. In *Conference on robot learning*. PMLR, 1–16. doi:10.48550/arXiv.1711.03938

[5] Nima Honarmand and Josep Torrellas. 2014. Replay debugging: Leveraging record and replay for program debugging. *ACM SIGARCH Computer Architecture News* 42, 3 (2014), 445–456. doi:10.1145/2678373.2665737

[6] Bernhard Jaeger, Kashyap Chitta, and Andreas Geiger. 2023. Hidden Biases of End-to-End Driving Models. In *Proc. of the IEEE International Conf. on Computer Vision (ICCV)*. doi:10.1109/ICCV51070.2023.00757

[7] Baharin A Jodat, Khouloud Gaaloul, Mehrdad Sabetzadeh, and Shiva Nejati. 2025. Automated Test Validators for Flaky Cyber-Physical System Simulators: Approach and Evaluation. *arXiv preprint arXiv:2508.20902* (2025). doi:10.48550/arXiv.2508.20902

[8] Olek Osikowicz, Phil McMinn, and Donghwan Shin. 2025. Empirically Evaluating Flaky Tests for Autonomous Driving Systems in Simulated Environments. In *2025 IEEE/ACM International Flaky Tests Workshop (FTW)*. 13–20. doi:10.1109/FTW66604.2025.00009

[9] Sanjeetha Pennada, Matthew Leach, Olek Osikowicz, and Donghwan Shin. 2026. *Replication Package of DT-Drive: A Tool for Deterministic Replay-Based Testing and Debugging of Autonomous Driving Systems*. doi:10.5281/zenodo.21631441

[10] Donghwan Shin and Sanjeetha Pennada. 2024. Towards Simplification of Failure Scenarios for Machine Learning-Enabled Autonomous Systems. In *2024 IEEE 24th International Conference on Software Quality, Reliability, and Security Companion (QRS-C)*. IEEE, 1089–1090. doi:10.1109/QRS-C63300.2024.00143

[11] H. Thane and H. Hansson. 2000. Using deterministic replay for debugging of distributed real-time systems. In *Proceedings 12th Euromicro Conference on Real-Time Systems. Euromicro RTS 2000.* 265–272. doi:10.1109/EMRTS.2000.854015

[12] Xinhai Zhang, Jianbo Tao, Kaige Tan, Martin Törngren, José Manuel Gaspar Sánchez, Muhammad Rusyadi Ramli, Xin Tao, Magnus Gyllenhammar, Franz Wotawa, Naveen Mohan, et al. 2022. Finding critical scenarios for automated driving systems: A systematic mapping study. *IEEE Transactions on Software Engineering* 49, 3 (2022), 991–1026. doi:10.1109/TSE.2022.3170122

Received 2026-05-12; accepted 2026-06-19